BASH

CHEAT SHEETS

A Complete Reference for Bash Scripting & the Command Line

Startup Files · Quoting · Redirection · Parameter Expansion · Arrays · Globbing · Tests · Loops · Functions · getopts · Traps & Debugging · Jobs · Prompt

27 REFERENCE SHEETS

CONTENTS

SETUP & CONFIGURATION

01 Introduction & Startup Files
02 Shell Options (set & shopt)

THE COMMAND LINE

03 Output, Quoting & Escaping
04 Redirection & Pipes
05 History & History Expansion
06 Readline, Key Bindings & Completion

VARIABLES & PARAMETERS

07 Variables & declare
08 Special Parameters & Environment
09 Parameter Expansion — Defaults & Length
10 Parameter Expansion — Substrings & Transforms

ARRAYS

11 Indexed Arrays
12 Associative Arrays

GLOBBING & EXPANSION

13 Globbing & Extended Globs
14 Brace, Tilde & Word Splitting

TESTS, ARITHMETIC & CONTROL FLOW

15 Test Expressions
16 if, case & select
17 Arithmetic
18 Loops

FUNCTIONS & SCRIPTING

19 Functions
20 Reading Input
21 Script Arguments & getopts
22 Errors, Traps & Debugging

PROCESSES & INTERACTIVE USE

23 Jobs, Subshells & Process Substitution
24 Aliases, Prompt & Navigation
25 Portability & Best Practices

QUICK REFERENCE

26 Quick Reference — Syntax & Expansion
27 Quick Reference — Scripting & Interactive

Table of Contents

Introduction & Startup Files

WHAT BASH IS

Bash (Bourne-Again SHell) was written by Brian Fox for the GNU Project in 1989 and is maintained by Chet Ramey. It is the default shell on most Linux distributions. macOS switched to zsh in Catalina but still ships **bash 3.2** at `/bin/bash`; get a current version with `brew install bash`. Which startup files run depends on whether the shell is a **login** shell and whether it is **interactive**.

GOOD TO KNOW

- Check your version: `bash --version` or `echo $BASH_VERSION`
- This set targets bash 5.x; features needing 4.0+ are tagged
- `$SHELL` is your login shell, not necessarily the current one
- To change shells: add the path to `/etc/shells`, then `chsh -s /path/to/bash`

HELLO WORLD & STARTUP CHECKS

```
#!/usr/bin/env bash
echo "Hello, World!"            # Hello, World!

bash --version | head -1        # GNU bash, version 5.2.37(1)-release ...
echo "$BASH_VERSION"            # 5.2.37(1)-release
echo "${BASH_VERSINFO[0]}"      # 5   (major version as a number)

# ~/.bash_profile: make login shells load ~/.bashrc too
if [ -f ~/.bashrc ]; then
  . ~/.bashrc
fi

[[ $- == *i* ]] && echo interactive   # i flag = interactive
shopt -q login_shell && echo login    # true in login shells

bash --norc --noprofile         # clean shell: no startup files
source ~/.bashrc                # reload after edits (same as: . ~/.bashrc)
```

CODE EXPLAINED

- → `#!/usr/bin/env bash` — shebang; finds the first `bash` on `PATH` (Homebrew bash beats `/bin/bash`)
- → `BASH_VERSINFO` — array: major, minor, patch… handy for version checks
- → `. ~/.bashrc` — login shells skip `.bashrc`, so source it from `.bash_profile`
- → `$-` — current single-letter flags; contains `i` when interactive
- → `--norc` / `--noprofile` — start without startup files to debug your config
- → `source` — runs a file in the **current** shell, so its variables stick

STARTUP FILE ORDER

SHELL TYPE	READS
Interactive login	`/etc/profile`, then the first found of `~/.bash_profile`, `~/.bash_login`, `~/.profile`
Interactive non-login	`~/.bashrc` (many distros read `/etc/bash.bashrc` first)
Non-interactive (script)	only the file named in `$BASH_ENV`
Login shell exits	`~/.bash_logout`

WHAT GOES WHERE

FILE	PUT HERE
`~/.bash_profile`	PATH / exports, then source `.bashrc`
`~/.bashrc`	aliases, functions, prompt, `shopt`
`~/.profile`	POSIX env shared with other shells
`~/.bash_logout`	cleanup on logout
`~/.inputrc`	Readline keys (Sheet 06)

⚠ COMMON MISTAKES

- Aliases in `~/.bash_profile` only: Linux terminals open non-login shells, which read `~/.bashrc` instead
- Creating `~/.bash_profile` stops bash reading `~/.profile`; source it if you need both
- `#!/bin/bash` on macOS runs bash 3.2, so `declare -A` and `mapfile` fail
- Editing `~/.bashrc` and expecting the open terminal to change; `source` it or open a new shell

✓ TIPS & BEST PRACTICES

- Keep `~/.bash_profile` to two lines: exports plus `. ~/.bashrc`
- Put `[[ $- == *i* ]] || return` near the top of `~/.bashrc` to skip prompt setup in scripts
- Test login behavior with `bash -l`, and a clean slate with `env -i bash --norc`
- Gate new features: `(( BASH_VERSINFO[0] >= 4 )) || exit 1`

QUICK SUMMARY

- `$BASH_VERSION` — which bash is running
- `~/.bash_profile` — login shells
- `~/.bashrc` — interactive non-login shells
- `source file` — run in the current shell
- `#!/usr/bin/env bash` — portable shebang

Shell Options (set & shopt)

TWO OPTION SYSTEMS

Bash has two sets of on/off switches. **set** controls the POSIX options (single letters like –e, or long names with –o name): a minus turns an option **on** and a plus turns it **off**, which feels backwards at first. **shopt** controls bash-only options: –s sets, –u unsets. Options apply to the current shell and are inherited by subshells, so setting them inside a script never changes your interactive shell.

NOTES

- `set -o` / `shopt` with no args list every option and its state
- `$-` holds active single-letter flags; `$SHELLOPTS` / `$BASHOPTS` hold the long names
- `set -euo pipefail` (strict mode) is covered on Sheet 22
- Put interactive options in `~/.bashrc`, script options at the top of the script

SET & SHOPT IN ACTION

```
set -o                          # list set options with on/off
set -o noclobber                # same as: set -C
echo hi > notes.txt             # bash: notes.txt: cannot overwrite existing file
echo hi >| notes.txt            # >| overrides noclobber once
set +o noclobber                # + turns it back OFF

echo "$-"                       # himBHs   (flags active in this shell)
shopt                           # list all shopt options
shopt -s globstar nullglob      # enable two at once
shopt -u nullglob               # disable
shopt -q extglob && echo on     # -q: no output, just exit status
shopt -p globstar               # shopt -s globstar  (re-runnable line)

# Temporarily change set options inside a function (bash 4.4+)
f() { local -; set -f; echo *; }   # prints *  ; noglob reverts on return
```

CODE EXPLAINED

- → `set -o name` / `set +o name` — long-name form; easier to read than letters
- → `>|` — force a single overwrite while `noclobber` is on
- → `$-` — `h` hashall, `i` interactive, `m` job control, `B` brace expansion, `H` history
- → `shopt -q` — use in `if` to test an option without output
- → `shopt -p` — prints commands that recreate the current state; save and replay
- → `local -` — saves set options and restores them when the function returns

COMMON SET OPTIONS

OPTION	FLAG	EFFECT
errexit	-e	exit when a command fails
nounset	-u	error on unset variables
pipefail	—	pipeline fails if any stage fails
xtrace	-x	print each command before running
noclobber	-C	> won't overwrite files
noglob	-f	turn off filename globbing
allexport	-a	export every assigned variable
vi / emacs	—	line-editing mode

COMMON SHOPT OPTIONS

OPTION	EFFECT
globstar	** matches across directories (4.0+)
nullglob	unmatched glob expands to nothing
failglob	unmatched glob is an error
dotglob	globs also match dotfiles
extglob	?() *() +() @() !() patterns
nocaseglob	case-insensitive globbing
histappend	append to history file, don't overwrite
autocd	type a directory name to cd into it (4.0+)

⚠ COMMON MISTAKES

- Reading `set -e` as "off": in `set`, `-` means **on** and `+` means **off**
- Enabling `shopt -s extglob` inside a function that uses the patterns: it must be on before the code is parsed
- Turning on `nullglob` globally, then `ls *.txt` lists everything when nothing matches
- Expecting `set` in a script to persist after it exits; it only affects that process

✓ TIPS & BEST PRACTICES

- Use long names in scripts (`set -o pipefail`); they document themselves
- Save and restore state: `old=$(shopt -p nullglob) … eval "$old"`
- `shopt -s checkwinsize histappend` belongs in every interactive `~/.bashrc`
- Use `failglob` in scripts to catch typos in file patterns early

QUICK SUMMARY

- `set -o x` / `set +o x` — turn a POSIX option on / off
- `shopt -s x` / `shopt -u x` — turn a bash option on / off
- `shopt -q x` — test an option
- `local -` — scope set options to a function

Output, Quoting & Escaping

PRINTING AND QUOTING

`echo` is fine for simple text, but its options and escape handling differ between shells, so prefer **`printf`** for anything formatted. Quoting controls which expansions happen: **single quotes** keep everything literal, **double quotes** still expand `$var`, `$(cmd)` and backslash escapes while preventing word splitting and globbing, and **`$'…'`** (ANSI-C quoting) turns escapes like `\n` and `\t` into real characters.

NOTES

- `printf` reuses its format for extra arguments: `printf '%s\n' a b c`
- `printf` does not add a newline unless the format has `\n`
- You can't put a `'` inside single quotes, even escaped
- Send errors to stderr with `>&2` so pipelines stay clean

OUTPUT & QUOTING EXAMPLES

```
echo "Hello"                    # Hello
echo -n "no newline"            # no newline   (prompt follows on same line)
echo -e "a\tb\nc"               # a<TAB>b  then c on a new line
printf '%s\n' one two           # one / two   (format reused per argument)
printf '%-8s|%5d|\n' id 42      # id      |   42|
printf '%.2f %05d %x\n' 3.14159 42 255   # 3.14 00042 ff
printf -v line '%s=%s' key val  # store in $line instead of printing

name="World"
echo 'Hi $name'                 # Hi $name    (single: literal)
echo "Hi $name"                 # Hi World    (double: expands)
echo $'tab:\tend'               # tab:<TAB>end   (ANSI-C quoting)
echo "Cost: \$5"                # Cost: $5
echo 'it'\''s'                  # it's   (close, escaped ', reopen)
printf '%q\n' "a b'c"           # a\ b\'c   (shell-safe re-quoting)
echo "failed" >&2               # write to stderr
```

CODE EXPLAINED

- → `echo -n` / `-e` — no newline / interpret escapes (not portable)
- → `%-8s` — left-align in 8 chars; `%5d` right-align; `%05d` zero-pad
- → `printf -v var` — format into a variable; no subshell needed
- → `'…'\''…'` — the standard trick for a quote inside single quotes
- → `%q` — output that can be pasted back into a shell safely
- → `>&2` — redirect stdout to stderr for messages

QUOTE TYPES

FORM	EXPANDS	USE FOR
`'text'`	nothing	literal strings, regexes
`"text"`	`$var` `$(cmd)` `\`	almost every variable
`$'text'`	escapes only	tabs, newlines, `\e`
`\c`	one char literal	`\$` `\"` `\`
`unquoted`	everything + split	rarely what you want

ESCAPES ($'…', ECHO -E, PRINTF)

ESCAPE	MEANING
`\n` `\t`	newline, tab
`\\` `\'`	backslash, quote
`\e` `\033`	escape (colors)
`\a` `\r`	bell, carriage return
`\xHH`	byte in hex
`\uHHHH`	Unicode char (4.2+)

PRINTF FORMATS

SPEC	MEANING
`%s` `%d`	string, integer
`%f` `%.2f`	float, 2 decimals
`%x` `%o`	hex, octal
`%-10s`	left-align, width 10
`%b`	arg with escapes expanded
`%q` `%%`	shell-quoted, literal %

⚠ COMMON MISTAKES

- `echo $var` unquoted: runs of spaces collapse and `*` expands to filenames
- `echo -e` in `sh` scripts: dash prints `-e` literally; use `printf`
- `printf "$msg"` : a `%` in the data becomes a format code; use `printf '%s\n' "$msg"`
- Escaping `'` inside single quotes (`'it\'s'`) doesn't work; use `"it's"` or `$'it\'s'`

✓ TIPS & BEST PRACTICES

- Default to double quotes around every expansion: `"$var"` , `"$(cmd)"`
- Use single quotes for anything you pass to `grep` , `sed` , or `awk`
- `printf '%s\n' "${arr[@]}"` prints one array element per line
- Color: `printf '\e[31m%s\e[0m\n' red`

QUICK SUMMARY

- `printf '%s\n'` — safe, portable output
- `'…'` — fully literal
- `"…"` — expand, but no splitting or globbing
- `$'…'` — real tabs and newlines
- `>&2` — print to stderr

Redirection & Pipes

STREAMS AND WHERE THEY GO

Every process starts with three open **file descriptors**: `0` stdin, 1 stdout, 2 stderr. Redirections rewire them before the command runs, and are processed **left to right**, which is why `> file 2>&1` and `2>&1 > file` behave differently. A **pipe** `|` connects one command's stdout to the next command's stdin; each side runs in its own subshell.

NOTES

- `2>&1` means "make fd 2 point where fd 1 points **right now**"
- Redirections can appear anywhere on the line: `> out cmd arg` works
- Pipeline exit status is the **last** command's, unless `set -o pipefail`
- `${PIPESTATUS[@]}` holds every stage's status (Sheet 08)

REDIRECTION PATTERNS

```
cmd > out.txt           # stdout → file (truncate)
cmd >> out.txt          # stdout → file (append)
cmd < in.txt            # file → stdin
cmd 2> err.txt          # stderr → file
cmd > all.txt 2>&1      # both → file   (order matters)
cmd &> all.txt          # same thing, bash shorthand
cmd &>> all.txt         # append both (bash 4+)
cmd 2>/dev/null         # throw away errors
cmd1 | cmd2             # stdout of cmd1 → stdin of cmd2
cmd1 |& cmd2            # stdout + stderr → cmd2 (bash 4+)
cmd | tee log.txt       # see output AND save it

cat <<EOF               # here-doc: $vars expand
User: $USER
EOF
grep -c ERR <<< "$text" # here-string: a string as stdin
exec 3> log.txt; echo hi >&3; exec 3>&-   # open fd 3, write, close
```

CODE EXPLAINED

→ `> f 2>&1` — stdout to `f`, then stderr to the same place

→ `&>` — shorthand for the line above; bash-only

→ `/dev/null` — discards whatever is written to it

→ `tee` — copies stdin to a file and to stdout; `tee -a` appends

→ `<<EOF … EOF` — multi-line stdin; delimiter must be alone on its line

→ `exec 3>` — open a descriptor for the rest of the script; `3>&-` closes it

ORDER MATTERS

FORM	RESULT
`> f 2>&1`	both streams → f
`2>&1 > f`	stdout → f, stderr → terminal
`2>&1 | cmd`	both streams → cmd
`> f 2> f`	both truncate f: garbled output
`1>&2`	stdout → stderr (same as >&2)

HERE-DOCS & STRINGS

FORM	BEHAVIOR
`<<EOF`	expands `$vars` and `$(cmd)`
`<<'EOF'`	fully literal (quote the delimiter)
`<<-EOF`	strips leading **tabs** (not spaces)
`<<< "$s"`	one string, newline added
`> f <<EOF`	write a file from a here-doc

SPECIAL FILES

PATH	PURPOSE
`/dev/null`	discard / empty input
`/dev/stderr`	fd 2 by name
`/dev/stdin`	fd 0 by name
`/dev/tcp/h/p`	bash opens a TCP socket
`>| f`	overwrite despite noclobber

⚠ COMMON MISTAKES

- `cmd 2>&1 > file` expecting both in the file: stderr was copied to the terminal before stdout moved
- `sort f > f` empties `f`: the shell truncates before `sort` reads; write to a temp file, then `mv`
- `sudo echo x > /etc/f` fails: the redirect runs as you; use `echo x | sudo tee /etc/f`
- Indenting a here-doc's closing `EOF` with spaces: only `<<-` strips, and only tabs

✓ TIPS & BEST PRACTICES

- Redirect a whole block once: `{ cmd1; cmd2; } > out.log 2>&1`
- Log everything a script prints: `exec > >(tee -a run.log) 2>&1` at the top
- Use `<<'EOF'` for generated scripts or config so `$` stays literal
- `set -o pipefail` so `false | true` reports failure

QUICK SUMMARY

- `> >> <` — write, append, read
- `2>&1` — merge stderr into stdout
- `&>` — both streams to a file
- `<<EOF` / `<<<` — inline stdin
- `|` / `|&` — pipe stdout / both

Sheet 05

History & History Expansion

HISTSIZE · HISTCONTROL · !! · !$ · ^old^new · history · fc

COMMAND HISTORY

Interactive bash records every command in memory and writes it to `~/.bash_history` when the shell exits. **History expansion** (`!` shortcuts) rewrites the line before it runs, so `sudo !!` reruns the last command with `sudo`. By default each terminal overwrites the file on exit and older lines are lost; a few lines in `~/.bashrc` fix both problems.

SETUP AND SHORTCUTS

```
# ~/.bashrc
HISTSIZE=10000                  # lines kept in memory
HISTFILESIZE=20000              # lines kept in ~/.bash_history
HISTCONTROL=ignoreboth          # skip duplicates + lines starting with space
HISTIGNORE='ls:bg:fg:history'   # never record these
HISTTIMEFORMAT='%F %T  '        # show timestamps in `history`
shopt -s histappend             # append on exit instead of overwrite
PROMPT_COMMAND="history -a; $PROMPT_COMMAND"   # save after every command

history 5                       # last 5 commands
sudo !!                         # rerun last command with sudo
!grep                           # rerun most recent command starting "grep"
cp file.txt /tmp && cd !$:h     # !$ = last arg; :h = its directory
^teh^the                        # rerun previous, replacing teh → the
!42:p                           # print command 42 without running it
fc                              # open last command in $EDITOR, run on save
```

NOTES

- History expansion is interactive-only; it's off in scripts
- `!` inside double quotes still expands: `echo "hi!"` can fail
- `set +H` turns off `!` expansion entirely
- Add `:p` to any designator to print it instead of running it
- Ctrl-R searches history interactively (Sheet 06)

CODE EXPLAINED

- → `ignoreboth` — `ignorespace` + `ignoredups`; start a line with a space to keep it out
- → `HISTTIMEFORMAT` — any `strftime` format; also saves timestamps in the file
- → `history -a` in `PROMPT_COMMAND` — shares history across open terminals
- → `!$:h` — modifiers edit the recalled word (`:h` head, `:t` tail)
- → `^a^b` — quick substitution on the previous command, first match only
- → `fc` — fix command; `fc -l` lists, `fc -s` reruns

EVENT DESIGNATORS

FORM	RECALLS
`!!`	previous command
`!n`	command number n
`!-n`	n commands back
`!str`	latest starting with str
`!?str?`	latest containing str
`^a^b`	previous, a → b
`!!:gs/a/b/`	previous, every a → b

WORD DESIGNATORS

FORM	WORD(S)
`!^`	first argument
`!$`	last argument
`!*`	all arguments
`!:2`	word 2 (0 = command)
`!:2-4`	words 2 to 4
`!$:h !$:t`	dir part / file part
`!$:r !$:e`	drop / keep extension

HISTORY COMMAND

COMMAND	DOES
`history 20`	last 20 entries
`history -c`	clear in-memory list
`history -d 42`	delete entry 42
`history -a`	append new lines to file
`history -n`	read lines added elsewhere
`history -w`	write whole list to file
`fc -l -10`	list last 10

⚠ COMMON MISTAKES

- No `histappend`: with two terminals open, whichever closes last overwrites the other's history
- `HISTFILESIZE` smaller than `HISTSIZE` truncates the file on every exit
- `echo "Done!"` fails with "event not found"; use single quotes or `set +H`
- Typing secrets on the command line: they land in the history file; prefix with a space when `ignorespace` is set

✓ TIPS & BEST PRACTICES

- Press Alt-. to paste the last argument; faster than typing `!$`
- Check before running: add `:p`, then `!!` to run it
- `shopt -s histverify` puts expanded `!` commands back on the line for review
- `shopt -s cmdhist` keeps multi-line commands as one entry (on by default)

QUICK SUMMARY

- `HISTSIZE` / `HISTFILESIZE` — memory / file size
- `HISTCONTROL=ignoreboth` — skip dups and space-prefixed
- `!!` `!$` `!*` — previous command / last arg / all args
- `^old^new` — fix a typo and rerun
- `history -a` + `histappend` — keep everything

Readline, Key Bindings & Completion

THE LINE EDITOR

Bash uses the **GNU Readline** library to edit the command line. Its default key bindings come from Emacs, or vi with `set -o vi`. Readline settings live in `~/.inputrc` and apply to every Readline program (bash, `python`, `psql`…), while `bind` changes them for the current shell. **Programmable completion** (`complete`, `compgen`) controls what Tab offers; the `bash-completion` package adds rules for hundreds of commands.

NOTES

- Reload `~/.inputrc` without restarting: Ctrl-X Ctrl-R
- `bind -p` lists every binding; `bind -l` lists function names
- Install bash-completion: `brew install bash-completion@2` or `apt install bash-completion`

~/.INPUTRC, BIND & COMPLETE

```
# ~/.inputrc
set completion-ignore-case on
set show-all-if-ambiguous on      # list matches on the first Tab
"\e[A": history-search-backward   # Up arrow: search by typed prefix
"\e[B": history-search-forward    # Down arrow

# In bash / ~/.bashrc
set -o vi                         # vi-style editing (default: emacs)
bind '"\C-x\C-r": re-read-init-file'
bind -x '"\C-g": git status'      # run a shell command from a key

complete -W "start stop restart" svc    # fixed word list for `svc`
_deploy() {
  local cur=${COMP_WORDS[COMP_CWORD]}   # word being completed
  COMPREPLY=($(compgen -W "dev staging prod" -- "$cur"))
}
complete -F _deploy deploy        # use the function for `deploy`
```

CODE EXPLAINED

- → `history-search-backward` — type `git`, press Up, see only `git` commands
- → `\e[A` — the escape sequence most terminals send for Up
- → `bind -x` — bind a key to a shell command instead of an editing function
- → `COMP_WORDS` / `COMP_CWORD` — the words on the line and the current index
- → `compgen -W list -- "$cur"` — prints the words in `list` that start with `$cur`
- → `COMPREPLY` — the array of candidates your function returns

EDITING KEYS (EMACS MODE)

KEYS	ACTION
`Ctrl-A / Ctrl-E`	start / end of line
`Alt-B / Alt-F`	back / forward one word
`Ctrl-W`	delete word before cursor
`Ctrl-U / Ctrl-K`	delete to start / end
`Ctrl-Y`	paste last deleted text
`Ctrl-_`	undo

HISTORY & MISC KEYS

KEYS	ACTION
`Ctrl-R`	reverse search history
`Ctrl-G`	cancel search
`Ctrl-P / Ctrl-N`	previous / next command
`Alt-.`	insert last argument
`Ctrl-X Ctrl-E`	edit line in `$EDITOR`
`Tab Tab`	list completions

COMPLETION BUILTINS

COMMAND	MEANING
`complete -W`	complete from a word list
`complete -F fn`	call a function
`complete -f / -d`	files / directories
`complete -o default`	fall back to files
`compgen -c`	list all command names
`COMPREPLY`	results array

⚠ COMMON MISTAKES

- Putting `bind` lines in `~/.inputrc`, or `set completion-…` lines in `~/.bashrc`: each file has its own syntax
- Ctrl-S freezes the terminal (XOFF) instead of searching forward; run `stty -ixon` first
- Writing `COMPREPLY=$(compgen …)` without parentheses gives one string, not an array
- Leaving out `--` in `compgen -W list -- "$cur"`: a word starting with `-` is read as an option

✓ TIPS & BEST PRACTICES

- The prefix-search Up/Down binding is the single biggest speedup
- Ctrl-X Ctrl-E is the easiest way to write a long one-liner
- Add `set colored-stats on` to color completions like `ls --color`
- Find a key's escape code: press Ctrl-V, then the key

QUICK SUMMARY

- `~/.inputrc` — Readline settings for all programs
- `bind` — change bindings in this shell
- `set -o vi` — vi editing mode
- `complete -W` / `-F` — define Tab completion
- `compgen` — generate matching candidates

Variables & declare

VARIABLES

Assign with `name=value` and **no spaces** around =; read with `$name` or `${name}`. All bash variables are strings unless you give them an attribute with **`declare`** (`–i` integer, `–a` array, `–r` read-only…). Variables are global to the script by default; `local` limits one to a function, and `export` passes it to child processes.

RULES

- Names: letters, digits, `_` ; can't start with a digit; case-sensitive
- Convention: `UPPER` for exported/env vars, `lower` for script vars
- No `$` on the left side: `count=1` , not `$count=1`
- An unset variable expands to an empty string (unless `set -u`)

ASSIGNING & DECLARING

```
name="Ada"                    # no spaces around =
echo "$name ${name}s"         # Ada Adas   (braces mark where the name ends)
readonly PI=3.14159           # constant; reassigning is an error
export EDITOR=vim             # visible to child processes
DEBUG=1 ./build.sh            # set for this one command only

declare -i n=5; n+=3          # 8     (-i: assignments are arithmetic)
declare -l lower="HeLLo"      # hello (-u for UPPER; bash 4+)
declare -p name               # declare -- name="Ada"
declare -n ref=name           # nameref (4.3+): ref is another name for name
ref="Grace"; echo "$name"     # Grace

f() { local count=0; count+=1; echo "$count"; }   # local to f
f; echo "${count-unset}"      # 01  then  unset
unset name                    # remove the variable
```

CODE EXPLAINED

- → `${name}s` — braces needed when text follows the name
- → `VAR=x cmd` — temporary env var for one command; the shell's value is unchanged
- → `declare -i` — `n+=3` adds instead of appending text
- → `declare -p` — prints the type and value; great for debugging
- → `count+=1` without `-i` appends: `0` becomes `01`
- → `${count-unset}` — default if unset (Sheet 09)

DECLARE FLAGS

FLAG	MEANING
`-a`	indexed array
`-A`	associative array (4.0+)
`-i`	integer (arithmetic assign)
`-l / -u`	force lower / upper case
`-r`	read-only
`-x`	export to children
`-n`	nameref (4.3+)
`-g`	global, even inside a function

SCOPE

FORM	VISIBLE TO
`x=1`	whole script (global)
`local x=1`	function + functions it calls
`export X=1`	this shell + child processes
`X=1 cmd`	only that command
`( x=1 )`	subshell only; lost after
`x=1 | cmd`	lost: each pipe side is a subshell

ASSIGNMENT FORMS

FORM	RESULT
`x=hello world`	runs `world` with x=hello
`x = 1`	runs command x with args
`x="a b"`	a b
`x=$(date +%F)`	command output
`x+=more`	append text
`x=`	set but empty

⚠ COMMON MISTAKES

- `x = 1` runs a command named `x` ; bash assignments allow no spaces
- `x=hello world` sets `x` only for running a command named `world` ; quote values with spaces
- Forgetting `local` in functions: the variable silently overwrites a global of the same name
- Setting a variable in a pipeline (`cmd | while read …; do x=…`) and expecting it afterwards

✓ TIPS & BEST PRACTICES

- Always declare function variables with `local`
- Use `readonly` for config constants at the top of a script
- `declare -p var` shows exactly what a variable holds, including quoting
- `export -n VAR` stops exporting without unsetting

QUICK SUMMARY

- `name=value` — assign, no spaces
- `"$name"` — read, always quoted
- `local` / `export` / `readonly` — scope and protection
- `declare -i -a -A -n` — give a variable a type
- `unset name` — remove it

Special Parameters & Environment

BUILT-IN PARAMETERS

Bash sets a group of **special parameters** you can read but not assign: the script's arguments, the last exit status, process IDs, and more. Alongside them are **shell variables** bash maintains (`RANDOM`, `SECONDS`, `LINENO`) and **environment variables** inherited from the parent process (`PATH`, `HOME`, `USER`). The most important rule: use **`"$@"`** (quoted) to pass arguments along unchanged.

NOTES

- `"$@"` expands to one word per argument; `"$*"` joins them into one
- `$?` is overwritten by every command; save it right away
- Inside a function, `$1`, `$#`, `$@` refer to the function's arguments
- `env` / `printenv` list exported variables; `set` lists everything

READING SPECIAL PARAMETERS

```
# run as:  ./demo.sh one "two words" three
echo "$0"                    # ./demo.sh
echo "$1 | $2"               # one | two words
echo "$#"                    # 3
for a in "$@"; do echo "[$a]"; done   # [one] [two words] [three]
echo "$*"                    # one two words three   (single string)
echo "${10}"                 # 10th argument needs braces

false; echo "$?"             # 1   (exit status of last command)
echo "$$ $BASHPID"           # PID of script / of the current (sub)shell
sleep 30 & echo "$!"         # PID of the last background job
false | true; echo "${PIPESTATUS[@]}"   # 1 0
echo "$RANDOM"               # 0-32767
echo "$SECONDS $EPOCHSECONDS" # secs since start / Unix time (5.0+)
echo "$LINENO ${BASH_SOURCE[0]}"        # current line / this file
```

CODE EXPLAINED

- → `$0` — how the script was called (path as typed)
- → `"$@"` — preserves `two words` as one argument
- → `${10}` — `$10` means `${1}` followed by `0`
- → `$?` — 0 is success; 1–255 is failure
- → `$BASHPID` — differs from `$$` inside `( … )` subshells
- → `PIPESTATUS` — array of every stage's exit status in the last pipeline

POSITIONAL

PARAM	VALUE
`$0`	script name
`$1 … $9`	arguments 1–9
`${10}`	argument 10+
`$#`	argument count
`"$@"`	all args, separately
`"$*"`	all args, one string
`shift`	drop $1, renumber

STATUS & PROCESS

PARAM	VALUE
`$?`	last exit status
`$$`	PID of the shell
`$!`	PID of last & job
`$BASHPID`	PID of current subshell
`$_`	last arg of previous cmd
`$-`	current shell flags
`PIPESTATUS`	statuses of last pipeline

COMMON ENVIRONMENT

VARIABLE	HOLDS
`PATH`	command search dirs, `:` separated
`HOME`	your home directory
`PWD / OLDPWD`	current / previous dir
`USER / UID`	user name / numeric ID
`IFS`	word-split characters
`SRANDOM`	32-bit random (5.1+)
`BASH_SOURCE`	file of current code

⚠ COMMON MISTAKES

- `for a in $@` (unquoted): arguments with spaces are split into separate words
- `if [ $? -eq 0 ]` after an `echo` you added for debugging: `$?` is now echo's status
- `$10` reads as `$1` followed by a literal `0`; use `${10}`
- `PATH=/my/bin` replaces the whole search path; use `PATH="/my/bin:$PATH"`

✓ TIPS & BEST PRACTICES

- Test commands directly: `if grep -q x f; then` instead of checking `$?`
- Save status before anything else: `cmd; rc=$?`
- `$(basename "$0")` gives a clean script name for usage messages
- Script directory: `dir=$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)`

QUICK SUMMARY

- `"$@"` — pass arguments through unchanged
- `$#` — how many arguments
- `$?` — exit status of the last command
- `$$` / `$!` — this shell's PID / last background PID
- `PIPESTATUS` — every stage's status

Parameter Expansion — Defaults & Length

DEFAULTS, ERRORS & LENGTH

Parameter expansion lets you supply a fallback, assign a default, abort with a message, or substitute a value, all without an `if`. The **colon** matters: `${v:-d}` uses the default when v is **unset or empty**, while `${v-d}` uses it only when v is **unset**. `${#v}` gives a length, and `${!v}` reads a variable whose **name** is stored in v.

NOTES

- `:` is a no-op command; `: "${X:=1}"` assigns without running anything
- `${v:?msg}` exits a non-interactive script with status 1
- `:=` can't assign to positional params: `${1:=x}` is an error
- For indirection, a nameref (`declare -n`) is often clearer than `${!v}`

DEFAULTS & LENGTH IN PRACTICE

```
unset u; e=""; s="value"
echo "${u:-def} ${e:-def} ${s:-def}"   # def def value
echo "${u-def} [${e-def}]"             # def []    (no colon: only unset)
: "${PORT:=8080}"; echo "$PORT"        # 8080  (assigns if unset or empty)
echo "${s:+yes} [${e:+yes}]"           # yes []    (alternate if non-empty)
: "${API_KEY:?must be set}"            # bash: API_KEY: must be set  (exits)

name=${1:-world}                       # first arg, or "world"
out=${OUTPUT_DIR:-$HOME/out}           # defaults can expand too

echo "${#s}"                           # 5   (string length)
arr=(a bb ccc); echo "${#arr[@]} ${#arr[2]}"   # 3 3

var=s;  echo "${!var}"                 # value   (indirect: $s)
APP_HOST=x APP_PORT=y
echo "${!APP_@}"                       # APP_HOST APP_PORT   (names by prefix)
```

CODE EXPLAINED

- → `:-` — use a default, don't store it
- → `:=` — use a default **and** assign it to the variable
- → `:?` — required variable; prints the message to stderr and exits
- → `:+` — the opposite of `:-` : value only when the variable is set
- → `${#arr[@]}` — element count; `${#arr[2]}` — length of element 2
- → `${!prefix@}` — names of all variables starting with the prefix

BEHAVIOR BY STATE

FORM	V UNSET	V=""	V="X"
`${v:-d}`	d	d	x
`${v-d}`	d	(empty)	x
`${v:=d}`	d, assigns	d, assigns	x
`${v=d}`	d, assigns	(empty)	x
`${v:?m}`	error	error	x
`${v?m}`	error	(empty)	x
`${v:+a}`	(empty)	(empty)	a
`${v+a}`	(empty)	a	a

LENGTH & INDIRECTION

FORM	RESULT
`${#v}`	length of string
`${#arr[@]}`	number of elements
`${#arr[i]}`	length of element i
`${!v}`	value of var named by v
`${!pre@}`	var names starting `pre`
`${!arr[@]}`	array indices / keys
`${v+set}`	"set" if v exists

⚠ COMMON MISTAKES

- Using `${v-d}` when the variable may be set but empty: no colon means empty counts as set
- `${PORT:=8080}` on its own line runs `8080` as a command; prefix it with `:`
- `${#arr}` gives the length of element 0, not the element count; use `${#arr[@]}`
- Testing existence with `[ -n "$v" ]` : that's false for set-but-empty; use `${v+x}` or `[[ -v v ]]`

✓ TIPS & BEST PRACTICES

- Required settings at the top of a script: `: "${DB_URL:?}"` documents and enforces them
- Optional args: `verbose=${2:-false}` reads cleaner than an `if`
- Build optional flags: `cmd ${DRY_RUN:+--dry-run}` adds the flag only when set
- Prefer `declare -n` over `${!v}` when you need to write through the reference

QUICK SUMMARY

- `${v:-d}` — default value
- `${v:=d}` — default and assign
- `${v:?msg}` — fail if missing
- `${v:+a}` — alternate if present
- `${#v}` / `${!v}` — length / indirection

Parameter Expansion — Substrings & Transforms

STRING SURGERY WITHOUT SUBPROCESSES

Bash can slice, trim, replace and change case inside `${…}` without calling `sed`, `cut` or `basename`, which is much faster in loops. `#` and `%` remove a **glob pattern** from the start or end; doubling them (`##`, `%%`) makes the match **greedy**. `/` replaces the first match and `//` replaces all. Patterns are globs, not regexes.

NOTES

- Memory aid: `#` is left of `$` on the keyboard (start); `%` is right (end)
- Negative offsets need a space or parens: `${v: -3}` or `${v:(-3)}`
- Case operators `^` `,` need bash 4+; `@U` `@L` `@u` need 5.1
- All forms work on every element of an array: `${arr[@]%.txt}`

SLICING, TRIMMING, REPLACING

```
f="/home/ada/report.final.txt"
echo "${f:6:3}"                 # ada    (offset 6, length 3)
echo "${f: -3}"                 # txt    (last 3 chars)
echo "${f##*/}"                 # report.final.txt   (basename)
echo "${f%/*}"                  # /home/ada          (dirname)
echo "${f%.*}"                  # /home/ada/report.final   (drop last ext)
echo "${f%%.*}"                 # /home/ada/report   (drop all exts)
b=${f##*/}; echo "${b#*.}"      # final.txt   (shortest prefix to first .)

s="a-b-c"
echo "${s/-/+} ${s//-/+}"       # a+b-c a+b+c   (first / all)
echo "${s/#a/X} ${s/%c/Z}"      # X-b-c a-b-Z   (anchor start / end)
echo "${s//-}"                  # abc   (delete every -)

n="hello World"
echo "${n^} | ${n^^} | ${n,,}"  # Hello World | HELLO WORLD | hello world
echo "${n@Q}"                   # 'hello World'   (quoted for reuse, 4.4+)
```

CODE EXPLAINED

- → `${f:6:3}` — substring; offset is 0-based
- → `##*/` — greedy: strips through the **last** `/`
- → `%/*` — non-greedy: strips from the last `/` to the end
- → `%.*` vs `%%.*` — one extension vs everything after the first dot
- → `/#` and `/%` — replace only at the start / end
- → `^` / `,` — first char; doubled = whole string

TRIM & REPLACE

FORM	EFFECT
`${v:o}`	from offset o to end
`${v:o:n}`	n chars from offset o
`${v#p}` / `${v##p}`	remove shortest / longest prefix
`${v%p}` / `${v%%p}`	remove shortest / longest suffix
`${v/p/r}`	replace first match
`${v//p/r}`	replace all matches
`${v//p}`	delete all matches

CASE & TRANSFORMS

FORM	EFFECT
`${v^}` / `${v^^}`	upper first / all (4.0+)
`${v,}` / `${v,,}`	lower first / all (4.0+)
`${v~~}`	toggle case of all
`${v@U}` / `${v@L}`	upper / lower all (5.1+)
`${v@Q}`	quoted for shell reuse (4.4+)
`${v@E}`	expand `\n` style escapes
`${v@A}`	as a `declare` statement

⚠ COMMON MISTAKES

- `${v:-3}` is the **default** operator, not "last 3 chars"; use `${v: -3}`
- Expecting regex: `${v//[0-9]+/}` matches a digit then a literal `+`; use `extglob` `+([0-9])` or `[[ =~ ]]`
- Using `${f%.*}` on `archive.tar.gz` and expecting `archive`; that needs `%%`
- Using `${v^^}` under `#!/bin/bash` on macOS: bash 3.2 has no case operators

✓ TIPS & BEST PRACTICES

- Replace `$(basename "$f")` with `${f##*/}` and `$(dirname "$f")` with `${f%/*}` in loops
- Change every extension at once: `mv "$f" "${f%.jpeg}.jpg"`
- Case-insensitive compare: `[[ ${a,,} == "${b,,}" ]]`

QUICK SUMMARY

- `${v:o:n}` — substring
- `${v#p}` / `${v%p}` — trim start / end
- `${v/p/r}` / `${v//p/r}` — replace first / all
- `${v^^}` / `${v,,}` — upper / lower
- `${v@Q}` — safely quoted copy

Indexed Arrays

ARRAYS OF VALUES

An indexed array holds an ordered list of strings with **0-based** integer indices. Arrays are the right way to store lists of filenames or arguments, because each element keeps its spaces. Always expand the whole array as **"${arr[@]}"**, quoted, to get one word per element. Arrays can be **sparse**: deleting an element leaves a gap in the indices.

NOTES

- `$arr` alone means `${arr[0]}`, a common surprise
- Arrays can't be exported to child processes
- Bash has no nested arrays; use naming schemes or associative arrays
- `declare -a arr` is optional; assignment creates the array

CREATING & USING ARRAYS

```
fruits=(apple "star fruit" cherry)
echo "${fruits[0]}"              # apple    (0-based)
echo "${fruits[-1]}"             # cherry   (negative = from the end)
echo "${#fruits[@]}"             # 3
fruits+=(date)                   # append one or more
fruits[10]=fig                   # sparse: indices 4-9 don't exist
echo "${!fruits[@]}"             # 0 1 2 3 10   (indices in use)
echo "${fruits[@]:1:2}"          # star fruit cherry   (slice)
unset 'fruits[1]'                # remove one element (no reindexing)
fruits=("${fruits[@]}")          # reindex to 0..n-1

for f in "${fruits[@]}"; do echo "[$f]"; done   # one per element
printf '%s\n' "${fruits[@]}"     # same, one per line

files=(*.log)                    # glob results into an array
read -ra parts <<< "a b c"       # split a string into an array
mapfile -t lines < notes.txt     # one element per line (4.0+)
```

CODE EXPLAINED

- → `(a "b c")` — quotes keep `b c` as one element
- → `+=( )` — append; `arr+=x` without parens appends to element 0's text
- → `${!arr[@]}` — list of indices, useful for sparse arrays
- → `${arr[@]:start:count}` — slice by position
- → `unset 'arr[1]'` — quote it so `[1]` isn't treated as a glob
- → `read -ra` — `-a` fills an array, `-r` keeps backslashes

EXPANSION FORMS

FORM	RESULT
`"${a[@]}"`	each element as its own word ✔
`"${a[*]}"`	one string, joined by 1st char of IFS
`${a[@]}`	elements re-split on spaces ✖
`$a`	element 0 only
`${a[i]}`	element i
`${#a[@]}`	element count

WHOLE-ARRAY OPERATIONS

FORM	EFFECT
`"${a[@]/x/y}"`	replace in every element
`"${a[@]%.txt}"`	trim every element
`"${a[@]^^}"`	uppercase every element
`(IFS=,; echo "${a[*]}")`	join with commas
`b=("${a[@]}")`	copy an array
`unset a`	delete the whole array

⚠ COMMON MISTAKES

- `for x in ${arr[@]}` unquoted: elements with spaces are split, and `*` expands to filenames
- `arr=$(ls)` creates a string, not an array; use `arr=(*)` or `mapfile -t arr < <(cmd)`
- Looping `for i in $(seq 0 $((${#a[@]}-1)))` over a sparse array hits missing indices; loop over `"${!a[@]}"`
- `words=($line)` splits on spaces **and** expands globs in the data

✓ TIPS & BEST PRACTICES

- Build command arguments in an array: `args=(-v --out "$dir")`, then `cmd "${args[@]}"`
- `mapfile -t arr < <(find . -name '*.md')` loads command output safely, one line per element
- Check for empty: `(( ${#arr[@]} == 0 ))`
- `declare -p arr` prints indices and values exactly

QUICK SUMMARY

- `arr=(a b c)` — create
- `"${arr[@]}"` — every element, safely
- `${#arr[@]}` — count
- `arr+=(x)` — append
- `${!arr[@]}` — indices

Associative Arrays

KEY-VALUE MAPS

An associative array maps **string keys** to string values, like a dictionary or hash map. It needs **bash 4.0+** and must be declared with **`declare -A`** before use; without it, bash treats the name as an indexed array and every key collapses to index 0. Key order is **not** preserved, so sort the keys yourself when order matters.

NOTES

- `local -A` declares one inside a function
- Keys can contain spaces: `h["new york"]=NY`
- Values are strings only; no nested maps
- Not available in macOS `/bin/bash` (3.2)

WORKING WITH A MAP

```
declare -A ages                 # required (bash 4.0+)
ages[alice]=31
ages[bob]=27
declare -A color=([red]=f00 [green]=0f0)    # literal form

echo "${ages[alice]}"           # 31
echo "${#ages[@]}"              # 2
echo "${!ages[@]}"              # bob alice   (keys, any order)
[[ -v ages[carol] ]] || echo "no carol"     # does the key exist?
(( ages[bob]++ ))               # 28   (arithmetic on a value)
unset 'ages[bob]'               # delete a key (quote it)

for k in "${!ages[@]}"; do      # iterate keys
  printf '%-6s %s\n' "$k" "${ages[$k]}"
done

declare -A count                # word frequency
for w in red blue red; do (( count[$w]++ )); done
echo "${count[red]}"            # 2
```

CODE EXPLAINED

- → `declare -A` — without it, `ages[alice]=31` sets index 0 of an indexed array
- → `[[ -v h[k] ]]` — true if the key exists, even with an empty value
- → `${!h[@]}` — keys; `${h[@]}` — values
- → `(( h[k]++ ))` — missing keys start at 0, so counting just works
- → `unset 'h[k]'` — quoting prevents glob expansion of `[k]`
- → `count[$w]` — the key can come from a variable

SYNTAX

FORM	MEANING
`declare -A h`	create empty map
`h[k]=v`	set a value
`"${h[k]}"`	get a value
`"${!h[@]}"`	all keys
`"${h[@]}"`	all values
`${#h[@]}`	number of keys
`${h[k]+x}`	"x" if key exists (any 4.x)

INDEXED VS ASSOCIATIVE

FEATURE	INDEXED	ASSOC
`Declare`	optional	–A required
`Keys`	integers	strings
`Order`	by index	unordered
`Negative index`	✔	✖
`Slicing`	✔	✖
`bash 3.2`	✔	✖

⚠ COMMON MISTAKES

- Skipping `declare -A` : string keys are evaluated as arithmetic, usually 0, so every key overwrites the same slot
- `h=([a]=1)` without `-A` silently makes an indexed array
- Expecting keys in insertion order; bash returns them in hash order
- `unset h[$k]` unquoted can glob-match a file named like `h1` and delete the wrong thing

✓ TIPS & BEST PRACTICES

- Sorted keys: `mapfile -t keys < <(printf '%s\n' "${!h[@]}" | sort)`
- Use a map as a set: `seen[$x]=1` , then `[[ -v seen[$x] ]]`
- Pass a map to a function with a nameref: `f() { local -n m=$1; …; }; f ages`
- Fail early on old bash: `(( BASH_VERSINFO[0] >= 4 ))`

QUICK SUMMARY

- `declare -A h` — create a map
- `h[key]=value` — set
- `"${!h[@]}"` — keys
- `[[ -v h[key] ]]` — key exists
- `unset 'h[key]'` — delete

Globbing & Extended Globs

`* ? [...] · [[:digit:]] · extglob · !(…) @(…) · ** · nullglo`

FILENAME PATTERNS

Before a command runs, bash replaces unquoted **glob patterns** with the matching filenames, sorted alphabetically. The command never sees the `*`; it sees the list of files. With `shopt -s extglob` you get regex-like operators such as `!(…)` ("anything except"), and `shopt -s globstar` makes `**` search subdirectories recursively.

NOTES

- No match: the pattern is passed through **literally** (`*.csv`) by default
- `*` doesn't match a leading `.` unless you write `.*` or set `dotglob`
- Quoting turns globbing off: `"*.txt"` is a literal string
- The same patterns work in `case` and in `[[ $x == pattern ]]`

GLOB PATTERNS IN USE

```
ls *.txt                    # every .txt in this directory
ls file?.log                # file1.log, fileA.log  (exactly one char)
ls [abc]*.md                # starts with a, b or c
ls [!0-9]*                  # doesn't start with a digit
ls *[[:upper:]]*            # contains an uppercase letter

shopt -s globstar           # bash 4.0+
ls **/*.py                  # .py files in all subdirectories
shopt -s nullglob           # no match → expands to nothing
for f in *.csv; do echo "$f"; done   # runs 0 times if no .csv

shopt -s extglob
ls !(*.bak)                 # everything except .bak files
ls *.@(jpg|png|gif)         # any one of these extensions
rm -- *+([0-9]).tmp         # names ending in digits, then .tmp
[[ $ans == @(y|yes) ]] && echo "confirmed"
```

CODE EXPLAINED

- → `?` — exactly one character; `*` — zero or more
- → `[!…]` — negated set (`[^…]` also works in bash)
- → `[[:upper:]]` — POSIX class; goes **inside** a bracket expression
- → `**/` — zero or more directories deep with `globstar`
- → `nullglob` — makes empty loops safe
- → `rm --` — ends options, so a file named `-rf` can't become a flag

BASIC WILDCARDS

PATTERN	MATCHES
`*`	any string, incl. empty
`?`	any one character
`[abc]`	one of a, b, c
`[a-z]`	one char in range
`[!x] / [^x]`	any char except x
`[[:alpha:]]`	letter (also digit, space, alnum)

EXTENDED GLOBS (EXTGLOB)

PATTERN	MATCHES
`?(p|q)`	zero or one of the patterns
`*(p|q)`	zero or more
`+(p|q)`	one or more
`@(p|q)`	exactly one
`!(p|q)`	anything except these
`*.@(c|h)`	`.c` or `.h` files

GLOB OPTIONS (SHOPT)

OPTION	EFFECT
`nullglob`	no match → nothing
`failglob`	no match → error
`dotglob`	include dotfiles
`nocaseglob`	ignore case
`globstar`	** recurses (4.0+)
`GLOBIGNORE`	patterns to exclude

⚠ COMMON MISTAKES

- `for f in *.csv` with no CSVs runs once with `f='*.csv'` ; enable `nullglob` or test `[[ -e $f ]]`
- Using regex syntax (`.*` , `+`) in a glob: `*.log.*` means "contains .log.", not "any characters"
- `ls *.txt` with thousands of files can hit "Argument list too long"; use `find … -exec … +`
- Turning on `extglob` inside the same function or `if` block that uses it: bash parses first, so it errors

✓ TIPS & BEST PRACTICES

- Put `shopt -s extglob` near the top of a script, on its own line
- Preview what a glob matches before `rm` : `echo rm *.tmp`
- Case-insensitive match one time: `[Rr][Ee]adme*`
- Count matches: `files=(*.log); echo ${#files[@]}` (with `nullglob`)

QUICK SUMMARY

- `* ? […]` — any / one / set
- `**/` — recursive with `globstar`
- `!(p) @(a|b)` — extglob operators
- `nullglob` — no match means no words
- Quote to turn globbing off

Brace, Tilde & Word Splitting

{a,b} · {1..10} · ~ · expansion order · IFS · word splitting

HOW A LINE BECOMES WORDS

Bash transforms every command line through a fixed sequence of **expansions** before running it. **Brace expansion** happens first, before variables, which is why `{1..$n}` doesn't work. After variables and command substitutions expand, any **unquoted** result is split on IFS and then globbed. Quoting stops both steps, which is why `"$var"` is the safe default.

NOTES

- Brace expansion makes strings; it doesn't check files exist (unlike globs)
- Default `IFS` is space, tab, newline
- Empty unquoted variables vanish entirely; quoted ones become `""`
- `set -f` disables globbing; `IFS=` disables splitting

EXPANSIONS & SPLITTING

```
echo {a,b,c}.txt              # a.txt b.txt c.txt
mkdir -p src/{lib,test,docs}  # three directories at once
cp config.yml{,.bak}          # cp config.yml config.yml.bak
echo {1..5} {a..e}            # 1 2 3 4 5 a b c d e
echo {01..10..3}              # 01 04 07 10   (zero-pad + step, 4.0+)
echo ~ ~root                  # /home/ada /root
cd ~-                         # previous directory ($OLDPWD)

line="a  b   c"
printf '[%s]' $line; echo     # [a][b][c]        (split on IFS)
printf '[%s]' "$line"; echo   # [a  b   c]       (quoted: one word)
IFS=: read -ra dirs <<< "$PATH"   # split PATH on colons
n=3; echo {1..$n}             # {1..3}   braces expand before $n
for ((i = 1; i <= n; i++)); do echo "$i"; done   # use this instead
x='*'; echo $x                # lists files: unquoted result is globbed
```

CODE EXPLAINED

- → `{,.bak}` — empty first item gives the name itself, then with `.bak`
- → `{01..10..3}` — leading zero pads every number; third field is the step
- → `~-` / `~+` — `$OLDPWD` / `$PWD`
- → `$line` unquoted — runs of spaces collapse to single separators
- → `IFS=:` before `read` — changes IFS only for that command
- → `echo $x` — word splitting **and** globbing apply to the value

EXPANSION ORDER

STEP	EXPANSION
1	brace {a,b} {1..9}
2	tilde ~ ~user
3	parameter, arithmetic, command subst. (left to right)
4	word splitting on IFS (unquoted only)
5	pathname (glob) expansion
6	quote removal

BRACE FORMS

FORM	RESULT
{a,b,c}	a b c
x{1,2}y	x1y x2y
{1..5}	1 2 3 4 5
{5..1}	5 4 3 2 1
{a..e..2}	a c e
{1..2}{a,b}	1a 1b 2a 2b

TILDE FORMS

FORM	BECOMES
~	$HOME
~user	user's home
~+	$PWD
~-	$OLDPWD
"~"	literal ~ (quoted)
~/x	$HOME/x

⚠ COMMON MISTAKES

- `{1..$n}` doesn't expand, because braces run before variables; use a C-style `for` or `seq`
- `"~/file"` in quotes stays a literal tilde; use `"$HOME/file"` or leave `~/` unquoted
- `rm $file` where the name has a space removes two different files (or fails)
- Changing `IFS` globally and forgetting to reset it breaks later code; scope it: `IFS=, read …`

✓ TIPS & BEST PRACTICES

- Quote every expansion unless you **want** splitting: `"$v"`, `"$(cmd)"`, `"${a[@]}"`
- Backup before editing: `cp file{,.orig}`; restore: `mv file{.orig,}`
- Need splitting on purpose? Use `read -ra arr <<< "$s"` instead of `$s`
- ShellCheck flags almost every unquoted variable (SC2086)

QUICK SUMMARY

- `{a,b}` / `{1..n}` — generate words (literal numbers only)
- `~` — home directory, unquoted only
- Order: brace → tilde → `$` → split → glob
- `IFS` — controls splitting
- `"$var"` — no splitting, no globbing

Test Expressions

TESTING CONDITIONS

A test is a command that exits 0 (true) or 1 (false). `[ ]` (also called `test`) is the POSIX version and is an ordinary command, so its arguments get split and globbed. **`[[ ]]`** is bash syntax: no word splitting, supports `&& || `, glob patterns with `==`, and regexes with `=~`. Use `[[ ]]` in bash scripts and `(( ))` for numbers.

[[ ]] VS [ ]

- `[[ $v == x ]]` is safe unquoted; `[ "$v" = x ]` needs quotes
- `[[ a && b ]]` vs `[ a ] && [ b ]` (`-a` / `-o` are deprecated)
- `< >` compare strings in `[[ ]]`; in `[ ]` they are redirections
- Spaces inside the brackets are required: `[[ -f x ]]`, not `[[-f x]]`

COMMON TESTS

```
f=notes.txt; s="hello"; n=7
[[ -f $f ]] && echo "regular file"
[[ -d /tmp && -w /tmp ]] && echo "writable dir"
[[ -z $empty ]] && echo "empty or unset"
[[ -n $s ]] && echo "non-empty"
[[ $s == h* ]] && echo "glob match"        # unquoted right side = pattern
[[ $s == "h*" ]] || echo "no match"         # quoted right side = literal
[[ $s =~ ^h(el+)o$ ]] && echo "${BASH_REMATCH[1]}"   # ell
[[ $n -gt 5 ]] && echo "numeric >"          # or: (( n > 5 ))
[[ -v HOME ]] && echo "HOME is set"
[[ new.log -nt old.log ]] && echo "newer"
[[ ! -e $f || -s $f ]] && echo "missing or non-empty"
[ "$s" = hello ] && echo "POSIX: quote vars, use single ="
re='^[0-9]+$'; [[ $n =~ $re ]] && echo "integer"
```

CODE EXPLAINED

- → `-z` / `-n` — empty / non-empty string
- → `==` with unquoted `h*` — glob pattern; quote it for a literal compare
- → `=~` — extended regex; groups land in `BASH_REMATCH`
- → `-gt` — numeric; `>` inside `[[ ]]` would compare as strings
- → `-v name` — variable is set (even if empty)
- → `re=…; [[ $x =~ $re ]]` — keep regexes in a variable, unquoted

FILE TESTS

TEST	TRUE IF
`-e f`	exists
`-f f`	regular file
`-d f`	directory
`-L f`	symbolic link
`-r / -w / -x f`	readable / writable / executable
`-s f`	exists, size > 0
`a -nt b`	a newer than b (`-ot` older)

STRING TESTS

TEST	TRUE IF
`-z s`	length is 0
`-n s`	length > 0
`a == b`	equal / glob match
`a != b`	not equal
`a < b`	sorts before
`a =~ re`	regex match
`-v name`	variable is set

INTEGER TESTS

TEST	MEANING
`-eq / -ne`	equal / not equal
`-lt / -le`	less / less-or-equal
`-gt / -ge`	greater / greater-or-equal
`(( a < b ))`	arithmetic form
`!`	negate
`&& / ||`	and / or in `[[ ]]`
`( … )`	group in `[[ ]]`

⚠ COMMON MISTAKES

- `[ $v = x ]` with empty `$v` becomes `[ = x ]`: "unary operator expected"
- `[[ $a > $b ]]` on numbers compares as text, so `10 > 9` is false; use `-gt` or `(( ))`
- Quoting the regex: `[[ $s =~ "^h.*" ]]` matches the literal characters `^h.*`
- `if [ -f $file ]` with a name containing spaces passes several words to `[`

✓ TIPS & BEST PRACTICES

- In bash scripts, use `[[ ]]` for strings and files, `(( ))` for numbers
- Store complex regexes in a variable to avoid quoting problems
- Test a command directly: `if grep -q …` is clearer than `[ $? -eq 0 ]`
- `help test` lists every operator

QUICK SUMMARY

- `[[ -f f ]]` / `[[ -d d ]]` — file / directory exists
- `[[ -z s ]]` / `[[ -n s ]]` — empty / non-empty
- `[[ s == pat ]]` — glob match
- `[[ s =~ re ]]` — regex, captures in `BASH_REMATCH`
- `-eq -lt -gt` — integer comparisons

if, case & select

BRANCHING

`if` runs a **command** and branches on its exit status; `[[ ]]` is just one such command, so `if grep -q …` works directly. **`case`** matches one value against glob patterns and is the cleanest way to handle subcommands and options. **`select`** prints a numbered menu, reads a choice, and loops until you `break`.

NOTES

- `then` needs a newline or `;` before it: `if cmd; then`
- `case` patterns are globs, tried top to bottom; first match wins
- `select` puts the chosen item in the variable and the typed number in `$REPLY`
- An empty `if` body is a syntax error; use `:` as a placeholder

IF & SELECT

```
if [[ -f config.yml ]]; then
  echo "using YAML"
elif [[ -f config.json ]]; then
  echo "using JSON"
else
  echo "no config" >&2
  exit 1
fi

if ! grep -q "ready" status.txt; then
  echo "not ready yet"
fi

PS3="Deploy to: "
select env in dev staging prod quit; do
  [[ $env == quit ]] && break
  [[ -n $env ]] && echo "→ $env (#$REPLY)"
done
```

CASE

```
case "$1" in
  start|up)
    echo "starting" ;;
  stop)
    echo "stopping" ;;
  *.tar.gz|*.tgz)
    tar xzf "$1" ;;
  -h|--help)
    usage ;;
  [0-9]*)
    echo "number: $1" ;;
  "")
    echo "missing argument" >&2 ;;
  *)
    echo "unknown: $1" >&2
    exit 1 ;;
esac
```

CONDITION FORMS

FORM	USE
`if cmd; then`	branch on exit status
`if ! cmd; then`	branch on failure
`if [[ … ]]; then`	strings, files, regex
`if (( … )); then`	numbers
`a && b`	run b only if a succeeds
`a || b`	run b only if a fails

CASE SYNTAX

FORM	MEANING
`a|b)`	either pattern
`*)`	default (match anything)
`"")`	empty value
`;;`	stop after this branch
`;&`	fall through to next body (4.0+)
`;;&`	keep testing patterns (4.0+)

SELECT

ITEM	ROLE
`PS3`	menu prompt
`$var`	chosen item text
`$REPLY`	what the user typed
`empty $var`	invalid number
`break`	leave the menu
`Ctrl-D`	EOF also exits

⚠ COMMON MISTAKES

- `a && b || c` as if/else: if `b` fails, `c` runs too; use a real `if` when `b` can fail
- Forgetting `;;` at the end of a `case` branch: syntax error near the next pattern
- Quoting a pattern in `case`: `"*.txt")` matches only the literal text `*.txt`
- Writing `if [[ … ]] then` without `;` or a newline before `then`

✓ TIPS & BEST PRACTICES

- Order `case` branches from specific to general; put `*)` last
- Use `case` instead of long `elif` chains on one variable
- One-liners: `[[ -d $d ]] || mkdir -p "$d"`
- Exit with a usage message in the `*)` branch

QUICK SUMMARY

- `if cmd; then … elif … else … fi` — branch on status
- `case $x in pat) … ;; esac` — match patterns
- `;&` / `;;&` — fall through / keep testing
- `select v in …; do … done` — numbered menu

Arithmetic

INTEGER MATH

Bash does **integer-only** math with signed 64-bit numbers. `$(( expr ))` substitutes the result; `(( expr ))` runs as a command and is **true when the result is non-zero**, so it doubles as a numeric test. Inside either form you can use C-style operators and drop the $ from variable names. For decimals, hand the work to `bc` or `awk`.

NOTES

- Division truncates toward zero: `$(( -7 / 2 ))` is `-3`
- Leading `0` means octal: `08` and `09` are errors
- Unset or non-numeric variables count as `0`
- Overflow wraps silently (64-bit)

ARITHMETIC EXAMPLES

```
a=17; b=5
echo $(( a + b ))              # 22
echo $(( a / b ))              # 3    (integer division)
echo $(( a % b ))              # 2    (remainder)
echo $(( a ** 2 ))             # 289
echo $(( (a + b) * 2 ))        # 44
(( a++ )); echo "$a"           # 18
(( b += 10 )); echo "$b"       # 15   (no $ needed inside (( )))
(( a > b )) && echo "a is bigger"
echo $(( a > b ? a : b ))      # 18   (ternary)
echo $(( 16#ff )) $(( 2#1010 ))    # 255 10   (base#number)
echo $(( 0x1F )) $(( 010 ))    # 31 8   (hex, and octal from leading 0)
echo $(( 10#09 ))              # 9    (force base 10)
printf '%.2f\n' "$(echo '10/3' | bc -l)"   # 3.33
awk 'BEGIN { printf "%.1f\n", 7 / 2 }'     # 3.5
```

CODE EXPLAINED

- → `$(( ))` — expression → number (substituted into the line)
- → `(( ))` — expression as a command: exit 0 if non-zero, 1 if zero
- → `**` — power; `%` — remainder
- → `16#ff` — any base from 2 to 64
- → `10#$n` — strip leading zeros from input like `08`
- → `bc -l` — arbitrary precision with the math library

OPERATORS (HIGH → LOW)

OPERATOR	MEANING
`++ --`	increment / decrement
`**`	power
`* / %`	multiply, divide, remainder
`+ -`	add, subtract
`<< >>`	bit shift
`< <= > >= == !=`	compare (1 or 0)
`& ^ |`	bitwise and, xor, or
`&& ||`	logical and, or
`?:`	ternary
`= += -= *= /=`	assignment

FORMS

FORM	USE
`$(( a + 1 ))`	value in a string or command
`(( a++ ))`	update a variable
`(( a > 3 ))`	test in `if` / `while`
`let "a = b * 2"`	older command form
`declare -i n`	assignments are arithmetic
`$[ a + 1 ]`	obsolete, don't use
`expr 1 + 2`	external, slow; avoid

⚠ COMMON MISTAKES

- `(( count++ ))` when `count` is 0 returns status 1, which kills a `set -e` script; use `(( ++count ))` or `count=$((count + 1))`
- Reading user input `08` or `09` as a number: octal error; use `$(( 10#$n ))`
- Expecting `$(( 10 / 3 ))` to be `3.33` ; bash has no floats
- Writing `$(( $a+$b ))` with spaces in values; inside `(( ))` plain names are cleaner

✓ TIPS & BEST PRACTICES

- Prefer `(( n > 5 ))` over `[ $n -gt 5 ]` for numeric tests
- Commas run several expressions: `(( a = 1, b = a + 2 ))`
- Random in a range: `$(( RANDOM % 6 + 1 ))` (dice roll)
- Round with printf: `printf '%.0f' 2.6` gives `3`

QUICK SUMMARY

- `$(( expr ))` — compute a value
- `(( expr ))` — compute and test
- `** % ?:` — power, remainder, ternary
- `16#ff` / `10#08` — explicit bases
- `bc` / `awk` — decimals

Loops

REPEATING COMMANDS

`for … in` walks a list of words (filenames, array elements, brace ranges). The C-style `for (( ))` counts with arithmetic. `while` repeats while a command succeeds, and `until` repeats while it fails. The standard way to read a file line by line is **`while IFS= read -r line`**, fed by a redirect or process substitution, not a pipe.

NOTES

- `for x; do` with no `in` loops over `"$@"`
- A loop's output can be redirected once: `done > out.txt`
- `break 2` / `continue 2` act on the enclosing loop
- `while :` or `while true` loops forever

LOOP PATTERNS

```
for f in *.jpg; do echo "converting $f"; done      # over a glob
for name in "${names[@]}"; do echo "$name"; done    # over an array
for i in {1..5}; do echo "$i"; done                 # fixed range
for ((i = 0; i < 10; i += 2)); do echo "$i"; done   # 0 2 4 6 8

while IFS= read -r line; do                  # file, line by line
  echo "> $line"
done < input.txt

count=0
while read -r user _; do (( ++count )); done < <(who)  # count survives
until curl -sf localhost:8080/health; do sleep 2; done  # retry until up

for f in *; do
  [[ -d $f ]] && continue                    # skip directories
  [[ $f == STOP ]] && break                  # leave the loop
done
```

CODE EXPLAINED

- `for f in *.jpg` — glob expands to a list; quote `"$f"` inside
- `{1..5}` — literal range; use `(( ))` when the end is a variable
- `IFS=` — keep leading/trailing spaces; `-r` — keep backslashes
- `< <(cmd)` — process substitution keeps the loop in this shell
- `read -r user _` — `_` swallows the rest of the line
- `until` — opposite of `while`; loops until success

LOOP FORMS

FORM	ITERATES OVER
`for x in a b c`	listed words
`for x in "${arr[@]}"`	array elements
`for ((i=0;i<n;i++))`	arithmetic counter
`while cmd`	while cmd succeeds
`until cmd`	until cmd succeeds

FEEDING A WHILE READ LOOP

FORM	NOTES
`done < file`	read a file
`done < <(cmd)`	command output, same shell
`done <<< "$var"`	lines of a variable
`cmd | while …`	**subshell**: vars lost
`IFS=, read -r a b`	split fields on commas

LOOP CONTROL

COMMAND	EFFECT
`break`	exit the loop
`break 2`	exit two levels
`continue`	next iteration
`continue 2`	next of outer loop
`while :`	infinite loop

⚠ COMMON MISTAKES

- `for line in $(cat file)` splits on spaces, not lines, and globs; use `while IFS= read -r`
- `cat f | while read …; do n=…; done` then `echo $n`: the loop ran in a subshell, so `n` is empty
- `for i in {1..$n}` doesn't expand; use `for ((i = 1; i <= n; i++))`
- `ssh` or `ffmpeg` inside a read loop eat its input; add `< /dev/null` to them

✓ TIPS & BEST PRACTICES

- Loop over files safely with globs, not `ls` output
- Read NUL-separated names from `find`: `while IFS= read -r -d '' f; do …; done < <(find . -print0)`
- Parallel-ish: append `&` to the loop body, then `wait` after `done`
- `shopt -s lastpipe` (scripts) runs the last pipe stage in the current shell

QUICK SUMMARY

- `for x in list` — words, globs, arrays
- `for (( ; ; ))` — counting
- `while IFS= read -r line` — lines of a file
- `< <(cmd)` — feed without a subshell
- `break` / `continue` — loop control

Functions

REUSABLE COMMANDS

A function is a named group of commands that you call like any other command, with its own arguments in $1, $2, "$@". Functions don't return values the way most languages do: **return** sets an exit status (0–255), and "returned data" is whatever the function **prints**, captured with $(fn). Variables are global unless declared **local**.

NOTES

- Define a function before the line that calls it
- Functions run in the current shell, so `cd` and variables persist
- Without `return`, the status is that of the last command
- `$0` is still the script name inside a function; use `$FUNCNAME`

DEFINING & CALLING

```
greet() {                          # POSIX-style definition
  local name=${1:-world}           # first argument, with a default
  echo "Hello, $name"
}
greet                              # Hello, world
greet Ada                          # Hello, Ada
msg=$(greet Bob)                   # capture output: "Hello, Bob"

is_even() { (( $1 % 2 == 0 )); }   # status of last command is returned
is_even 4 && echo even             # even

die() { echo "error: $*" >&2; exit 1; }
[[ -f app.cfg ]] || die "app.cfg missing"

split_name() {                     # write results into caller's vars
  local -n _first=$2 _last=$3      # namerefs (4.3+)
  _first=${1%% *}; _last=${1##* }
}
split_name "Ada Lovelace" f l; echo "$l, $f"   # Lovelace, Ada
```

CODE EXPLAINED

- → `${1:-world}` — default when no argument is passed
- → `$(greet Bob)` — runs in a subshell; only stdout comes back
- → `(( … ))` as the last command — makes the function a true/false test
- → `$*` in `die` — all arguments joined into one message
- → `exit` — ends the whole script; `return` ends just the function
- → `local -n` — writes go to the variable named by the argument

DEFINE & RETURN

FORM	MEANING
`f() { …; }`	POSIX definition
`function f { …; }`	bash keyword form
`return 3`	exit status 3
`return`	status of last command
`out=$(f)`	capture printed output
`f || echo failed`	test its status

INSIDE A FUNCTION

NAME	VALUE
`$1 $2 …`	function arguments
`$# / "$@"`	count / all arguments
`local v`	function-scoped var
`FUNCNAME`	call stack of names
`shift`	drop $1
`$?`	status of last call

INSPECT & MANAGE

COMMAND	DOES
`declare -f f`	print the definition
`declare -F`	list function names
`type f`	function, alias, or file?
`unset -f f`	delete a function
`export -f f`	make visible to `bash -c`
`command ls`	skip a function named `ls`

⚠ COMMON MISTAKES

- `return "text"` errors: `return` only takes a number 0–255; print the text instead
- `local out=$(cmd)` hides `cmd`'s failure (status comes from `local`); split it: `local out; out=$(cmd)`
- Debug `echo`s inside a function you capture with `$(…)` end up in the result; send them to `>&2`
- Calling a function before it's defined in the file: "command not found"

✓ TIPS & BEST PRACTICES

- Structure scripts as functions plus a final `main "$@"`
- Always pass arguments through with `"$@"`, never `$*`
- Return true/false with exit status so callers can write `if valid "$x"`
- Prefix nameref locals (`_first`) so they can't collide with the caller's names

QUICK SUMMARY

- `f() { …; }` — define
- `f arg1 arg2` — call
- `local` — keep variables private
- `return n` — status; `echo` — data
- `$(f)` — capture output

Reading Input

GETTING DATA IN

`read` takes one line from stdin, splits it on `IFS`, and assigns the pieces to variables; the **last variable gets the rest of the line**. Always use **`-r`** so backslashes are kept as typed. **`mapfile`** (also called `readarray`, bash 4+) loads a whole file or command output into an array, one line per element.

NOTES

- `read` returns non-zero at end of input, which ends `while read` loops
- Prompts from `read -p` go to stderr, so they show even when stdout is redirected
- With no variable name, `read` stores the line in `$REPLY`
- `read -e` uses Readline (arrow keys, Tab) for the input

READ & MAPFILE

```
read -r name                          # one line into $name
read -rp "Your name: " name           # with a prompt
read -rsp "Password: " pw; echo       # -s: don't echo what's typed
read -rn1 -p "Continue? [y/N] " ans   # return after one keypress
read -rt 5 -p "Quick! " a || echo "timed out"
read -r first rest <<< "a b c"        # first=a  rest="b c"
IFS=, read -r id name email <<< "7,Ada,ada@x.io"
read -ra words <<< "one two three"    # array: ${words[1]} = two
read -rei "main" -p "Branch: " br     # editable default (4.0+)

mapfile -t lines < todo.txt           # one element per line (4.0+)
echo "${#lines[@]} lines; first: ${lines[0]}"
mapfile -t -n 10 top < big.log        # first 10 lines only
mapfile -t users < <(cut -d: -f1 /etc/passwd)
data=$(< notes.txt)                   # whole file in one string
[[ -t 0 ]] || echo "stdin is a pipe or file"
```

CODE EXPLAINED

- → `-p` — prompt; `-s` — silent; `-n1` — single character
- → `-t 5` — give up after 5 seconds (status > 128)
- → `IFS=,` — split on commas for this `read` only
- → `-a` — fill an array instead of separate variables
- → `-e -i text` — pre-filled, editable answer
- → `mapfile -t` — strip the newline from each line

READ OPTIONS

OPTION	EFFECT
`-r`	keep backslashes (always use)
`-p str`	show a prompt
`-s`	silent (passwords)
`-t sec`	timeout
`-n N`	stop after N chars
`-d c`	stop at c, not newline
`-a arr`	split into an array
`-u fd`	read from file descriptor

MAPFILE OPTIONS

OPTION	EFFECT
`-t`	remove trailing newlines
`-n N`	read at most N lines
`-s N`	skip first N lines
`-O i`	start at index i
`-d c`	line delimiter (4.4+)
`-d ''`	NUL-separated input
`readarray`	same command, other name

YES/NO PATTERN

CODE	RESULT
`read -rp "OK? " a`	ask
`case $a in`	match the answer
`[Yy]*) …;;`	y, Y, yes, Yes…
`*) exit 1;;`	anything else
`esac`	end
`${a:-n}`	treat Enter as no

⚠ COMMON MISTAKES

- Leaving off `-r` : `C:\new` becomes `C:new`
- Leaving off `IFS=` when reading lines: leading and trailing spaces are stripped
- `echo "$x" | read y` sets `y` in a subshell; use `read y <<< "$x"`
- `mapfile` in a `#!/bin/bash` script on macOS: bash 3.2 doesn't have it

✓ TIPS & BEST PRACTICES

- Read lines exactly: `IFS= read -r line`
- Ask the terminal even when stdin is redirected: `read -r ans < /dev/tty`
- Use `$(< file)` instead of `$(cat file)` ; it skips starting `cat`
- Default answer: `read -rp "Port [8080]: " p; p=${p:-8080}`

QUICK SUMMARY

- `read -r var` — one line
- `read -rp "msg" var` — prompt
- `read -ra arr` — split into an array
- `IFS=, read -r a b` — split on commas
- `mapfile -t arr < file` — file to array

Script Arguments & getopts

`#!/usr/bin/env bash · shift · getopts · $OPTARG · $OPTIND ·`

PARSING COMMAND-LINE OPTIONS

getopts is the built-in parser for short options (`-v`, `-o file`, `-vo file`). Its option string lists the letters; a **colon after a letter** means it takes a value. It doesn't handle `--long` options, so for those write a `while`/`case` loop over `"$@"` and `shift` as you go. Either way, finish with only the positional arguments left in `"$@"`.

MAKING IT RUNNABLE

- Start with `#!/usr/bin/env bash`, then `chmod +x script.sh`
- Run it as `./script.sh`, or put its directory on `PATH`
- `--` marks the end of options: `rm -- -file.txt`
- Print usage to stderr and exit 2 on bad input

GETOPTS (SHORT OPTIONS)

```
usage() { echo "usage: ${0##*/} [-v] [-o file] in..." >&2; exit 2; }

verbose=0; out=out.txt
while getopts ":vo:h" opt; do
  case $opt in
    v)  verbose=1 ;;
    o)  out=$OPTARG ;;
    h)  usage ;;
    :)  echo "-$OPTARG needs a value" >&2; usage ;;
    \?) echo "unknown: -$OPTARG" >&2; usage ;;
  esac
done
shift $((OPTIND - 1))       # drop the parsed options
(( $# > 0 )) || usage       # need at least one input

for f in "$@"; do echo "$f → $out"; done
```

MANUAL LOOP (LONG OPTIONS)

```
while (( $# )); do
  case $1 in
    -v|--verbose)  verbose=1 ;;
    -o|--output)   out=$2; shift ;;
    --output=*)    out=${1#*=} ;;
    -h|--help)     usage ;;
    --)            shift; break ;;
    -*)            echo "unknown: $1" >&2
                   usage ;;
    *)             break ;;
  esac
  shift
done
# "$@" now holds only the positional args
echo "verbose=$verbose out=$out files=$#"
```

GETOPTS PIECES

ITEM	MEANING
`"vo:h"`	`-v`, `-o value`, `-h`
`:vo:h`	leading `:` = you handle errors
`$OPTARG`	the option's value
`$OPTIND`	index of next argument
`opt = ?`	unknown option
`opt = :`	missing value (silent mode)

POSITIONAL TOOLS

FORM	EFFECT
`shift`	drop $1
`shift 2`	drop $1 and $2
`set -- a b`	replace all args
`"$@"`	remaining args
`$#`	how many remain
`${@: -1}`	last argument

EXIT CODE CONVENTIONS

CODE	MEANING
`0`	success
`1`	general error
`2`	usage / bad arguments
`126`	found, not executable
`127`	command not found
`128+n`	killed by signal n (130 = Ctrl-C)

⚠ COMMON MISTAKES

- Forgetting `shift $((OPTIND - 1))`: the options are still in `"$@"` when you process files
- Expecting `getopts` to read `--verbose`; it only knows single letters
- `out=$2; shift` without checking `$2` exists: `-o` at the end silently gets an empty value
- Putting options after file names: `getopts` stops at the first non-option

✓ TIPS & BEST PRACTICES

- Use `"${0##*/}"` in usage text so it shows the script name, not its path
- Accept both `--output file` and `--output=file`, as shown
- Keep option defaults in variables at the top so `-h` can print them
- GNU `getopt` (external, with a t) handles long options but isn't on macOS by default

QUICK SUMMARY

- `getopts "ab:" opt` — short options
- `$OPTARG` — option value
- `shift $((OPTIND-1))` — leave only the args
- `while (( $# )); case $1` — long options
- `exit 2` — bad usage

Errors, Traps & Debugging

FAILING SAFELY

By default bash keeps going after a command fails. **Strict mode**, `set –euo pipefail`, makes a script stop on failures (`–e`), on unset variables (`–u`), and on failures anywhere in a pipeline (`pipefail`). `–e` has many exceptions, so treat it as a safety net, not error handling. **`trap`** runs code on exit or on signals, which is how you clean up temp files no matter how the script ends.

WHERE -E DOESN'T FIRE

- Commands tested by `if`, `while`, `&&`, `||` or `!`
- Any command except the last in a pipeline (without `pipefail`)
- Inside `$(…)` unless `shopt -s inherit_errexit` (4.4+)
- Inside a function called from an `if` or `&&`

A DEFENSIVE SCRIPT HEADER

```
#!/usr/bin/env bash
set -euo pipefail                       # strict mode
trap 'echo "line $LINENO: $BASH_COMMAND failed" >&2' ERR
tmp=$(mktemp -d)
trap 'rm -rf "$tmp"' EXIT               # runs on any exit
trap 'echo "interrupted" >&2; exit 130' INT TERM

grep -q x maybe.txt || true             # allowed to fail
if ! out=$(build 2>&1); then            # handle failure yourself
  echo "build failed: $out" >&2; exit 1
fi
name=${NAME:-anon}                      # -u safe: default for unset

PS4='+ ${BASH_SOURCE##*/}:${LINENO}: ' # richer trace prefix
set -x                                  # trace from here...
cp config.yml "$tmp/"
set +x                                  # ...to here
```

CODE EXPLAINED

- → `ERR` trap — runs when a command fails; `$BASH_COMMAND` is the culprit
- → `EXIT` trap — runs on normal exit, `exit`, `-e` abort, and most signals
- → `|| true` — marks a command as allowed to fail under `-e`
- → `if ! out=$(…)` — capture output and test status in one step
- → `PS4` — prefix for `set -x` lines; set it before tracing
- → `exit 130` — conventional status after Ctrl-C

STRICT-MODE OPTIONS

OPTION	EFFECT
`set -e`	exit on a failing command
`set -u`	unset variable is an error
`set -o pipefail`	pipe fails if any stage fails
`set -E`	ERR trap applies in functions
`inherit_errexit`	–e inside $(…) (4.4+)
`cmd || true`	opt out for one command

TRAP SIGNALS

SIGNAL	WHEN
`EXIT`	shell exits for any reason
`ERR`	a command fails
`INT`	Ctrl-C
`TERM`	`kill` (default signal)
`HUP`	terminal closed
`RETURN`	function/source returns
`trap - INT`	reset to default

DEBUGGING TOOLS

TOOL	USE
`bash -n s.sh`	syntax check, don't run
`bash -x s.sh`	trace every command
`set -x / set +x`	trace a section
`BASH_XTRACEFD=3`	send trace to fd 3
`caller`	print caller's line + file
`shellcheck s.sh`	static analysis
`trap '…' DEBUG`	run before each command

⚠ COMMON MISTAKES

- Relying on `-e` inside functions used in `if`: errors in them are silently ignored
- `local v=$(cmd)` or `export v=$(cmd)` always succeeds, so `-e` never sees `cmd` fail
- `(( i++ ))` with `i=0` returns 1 and kills a `set -e` script
- `set -o pipefail` plus `cmd | head -1` can fail with status 141 (SIGPIPE) when `cmd` keeps writing

✓ TIPS & BEST PRACTICES

- Set the `EXIT` trap right after creating a temp file
- Write messages to stderr and use distinct exit codes per failure
- Run `shellcheck` on every script; it catches most `-e` and quoting traps
- Trace only the part you're debugging with `set -x … set +x`

QUICK SUMMARY

- `set -euo pipefail` — stop on errors
- `trap 'cleanup' EXIT` — always clean up
- `trap '…' ERR` — report failures
- `set -x` + `PS4` — trace execution
- `bash -n` / `shellcheck` — check before running

Jobs, Subshells & Process Substitution

RUNNING THINGS CONCURRENTLY

A trailing & runs a command in the **background** and returns immediately; `wait` pauses until background jobs finish. Parentheses `( … )` run commands in a **subshell**, a forked copy where `cd` and variable changes don't leak out, while braces `{ …; }` group commands in the **current** shell. **Process substitution** `<(cmd)` turns a command's output into a filename, so tools that only accept files can read it.

NOTES

- Job specs: `%1` job 1, `%+` current, `%-` previous, `%vim` by name
- `$!` is the PID of the most recent background job
- Job control (`fg` , `bg` , Ctrl-Z) is for interactive shells
- Braces need spaces and a final `;` : `{ a; b; }`

BACKGROUND JOBS & GROUPING

```
./long_task.sh &                    # run in the background
echo "started PID $!"
jobs -l                             # list jobs with PIDs
fg %1                               # bring job 1 to the foreground
# Ctrl-Z suspends the foreground job, then:
bg %1                               # resume it in the background
disown %1                           # keep it alive after the terminal closes
nohup ./server.sh > srv.log 2>&1 &  # immune to hangup from the start

for h in web1 web2 web3; do ping -c1 "$h" > "$h.log" & done
wait                                # wait for all background jobs
wait -n                             # wait for whichever finishes next (4.3+)

( cd /tmp && make )                 # subshell: the cd doesn't leak out
{ echo header; cat data; } > out.csv   # group: one redirect, same shell
diff <(sort a.txt) <(sort b.txt)    # compare two command outputs
tar cf - src | tee >(sha256sum > sum.txt) > src.tar
```

CODE EXPLAINED

- → `jobs -l` — job numbers, PIDs and states
- → `disown` — removes the job from the shell's table so SIGHUP isn't sent
- → `wait` — returns when all children finish; `wait $pid` for one
- → `( … )` — changes stay inside; costs a fork
- → `<(cmd)` — becomes a path like `/dev/fd/63`
- → `>(cmd)` — a writable path whose data feeds `cmd`

JOB CONTROL

COMMAND	EFFECT
`cmd &`	start in background
`jobs`	list jobs
`fg %n`	to foreground
`bg %n`	resume in background
`Ctrl-Z`	suspend foreground job
`kill %n`	signal a job
`disown -h %n`	ignore HUP, keep in list

GROUPING & SUBSTITUTION

FORM	RUNS IN / GIVES
`( cmds )`	subshell
`{ cmds; }`	current shell
`$(cmd)`	output as a string
`<(cmd)`	output as a file path
`>(cmd)`	file path that feeds cmd
`coproc NAME { … }`	two-way pipe (4.0+)
`cmd1 | cmd2`	each side in a subshell

⚠ COMMON MISTAKES

- Exiting the terminal kills background jobs; use `disown` , `nohup` , or `tmux`
- Starting hundreds of `&` jobs at once; throttle with `wait -n` or use `xargs -P`
- `{ a; b }` without the final `;` (or newline) is a syntax error
- Using `cd` in a subshell and expecting the parent's directory to change

✓ TIPS & BEST PRACTICES

- Limit parallel jobs: `(( $(jobs -rp \| wc -l) >= 4 )) && wait -n` before each `&`
- Check each job's status: `wait "$pid" \|\| echo "$pid failed"`
- Use a subshell to scope `cd` , `IFS` or `set` changes temporarily
- `$( … )` nests cleanly; backticks don't, so avoid them

QUICK SUMMARY

- `cmd &` / `wait` — run in parallel, then sync
- `fg` / `bg` / `disown` — manage jobs
- `( … )` — isolated subshell
- `{ …; }` — group in the current shell
- `<(cmd)` — command output as a file

Aliases, Prompt & Navigation

MAKING THE SHELL YOURS

An **alias** is a text shortcut expanded when you type a command; for anything that needs arguments in the middle or logic, write a function instead. **PS1** defines the prompt using backslash escapes, and **PROMPT_COMMAND** runs before each prompt is drawn. The directory stack (`pushd`, `popd`) and `CDPATH` make moving between project folders fast.

NOTES

- Aliases are off in scripts unless `shopt -s expand_aliases`
- Wrap non-printing color codes in `\[ \]` or line wrapping breaks
- Put aliases and `PS1` in `~/.bashrc` (Sheet 01)
- `PROMPT_COMMAND` can be an array in bash 5.1+

~/.BASHRC CUSTOMIZATIONS

```
alias ll='ls -alF'
alias gs='git status'
alias rm='rm -i'                    # confirm before deleting
\rm temp.txt                        # backslash skips the alias once
command rm temp.txt                 # so does `command`
unalias gs                          # remove an alias
alias                               # list all aliases

PS1='\u@\h:\w\$ '                   # ada@laptop:~/src$
PS1='\[\e[32m\]\u\[\e[0m\]:\[\e[34m\]\w\[\e[0m\]\$ '   # green user, blue dir
PROMPT_COMMAND='printf "\e]0;%s\a" "${PWD##*/}"'       # dir in window title

pushd /etc > /dev/null              # cd, remembering where you were
popd                                # go back
cd -                                # toggle to the previous directory
CDPATH=.:~/projects                 # `cd api` finds ~/projects/api
```

CODE EXPLAINED

- → `\rm` / `command rm` — run the real command, skipping the alias
- → `\u \h \w` — user, short host name, working directory
- → `\[ … \]` — tells bash these bytes take no space on screen
- → `\e[32m` — ANSI color: 32 green, 34 blue, `0` reset
- → `\e]0;…\a` — escape sequence that sets the terminal title
- → `CDPATH` — extra directories `cd` searches for relative names

PS1 ESCAPES

ESCAPE	SHOWS
`\u` / `\h`	user / host
`\w` / `\W`	full / last dir
`\$`	# if root, else $
`\t` / `\A`	time 24h / HH:MM
`\d`	date
`\n`	newline
`\j` / `\!`	jobs / history number

ANSI CODES (\E[…M)

CODE	EFFECT
`0`	reset
`1` / `2`	bold / dim
`31` / `32`	red / green
`33` / `34`	yellow / blue
`35` / `36`	magenta / cyan
`1;32`	bold green
`38;5;208`	256-color orange

NAVIGATION

COMMAND	EFFECT
`cd` / `cd ~`	home
`cd -`	previous dir
`pushd dir`	push and cd
`popd`	pop and cd
`dirs -v`	numbered stack
`pushd +2`	rotate to entry 2
`shopt -s autocd`	type dir to cd

⚠ COMMON MISTAKES

- Aliases that take arguments mid-command (`alias g='grep $1 file'`); aliases only prepend text, so use a function
- Color codes without `\[ \]` : long commands wrap onto the same line and overwrite the prompt
- Defining an alias and using it on the same line, or in a function defined on that line: aliases expand when the line is read
- Overwriting `PROMPT_COMMAND` and losing what other tools (like `history -a`) added

✓ TIPS & BEST PRACTICES

- `type ll` shows whether a name is an alias, function, or file
- Use `tput setaf 2` instead of raw codes for portable colors
- Show the git branch with `__git_ps1` from git's `git-prompt.sh`
- Add to `PROMPT_COMMAND` instead of replacing it: `PROMPT_COMMAND="mycmd; $PROMPT_COMMAND"`

QUICK SUMMARY

- `alias name='cmd'` — shortcut
- `\cmd` — bypass an alias
- `PS1` — prompt; `\[ \]` around colors
- `PROMPT_COMMAND` — run before each prompt
- `pushd` / `popd` / `cd -` — move around

Portability & Best Practices

WRITING SCRIPTS THAT HOLD UP

Most shell bugs come from a short list: unquoted variables, parsing `ls`, ignored failures, and assuming a feature exists on every machine. Decide up front which shell you're targeting: use `#!/usr/bin/env bash` and bash features freely, or `#!/bin/sh` and stick to **POSIX**. Run **ShellCheck** on everything; it catches most of these problems automatically.

NOTES

- Install ShellCheck: `brew install shellcheck` / `apt install shellcheck`
- On Debian/Ubuntu, `/bin/sh` is **dash**, not bash
- Silence one warning: `# shellcheck disable=SC2086` above the line
- When a script passes about 100 lines, consider Python

A SCRIPT TEMPLATE

```
#!/usr/bin/env bash
# deploy.sh — sync a build to a server
set -euo pipefail

script_dir=$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)
tmp=$(mktemp -d)
trap 'rm -rf "$tmp"' EXIT

log() { printf '%s %s\n' "$(date +%T)" "$*" >&2; }

main() {
  command -v rsync > /dev/null || { log "rsync not found"; exit 127; }
  local target=${1:?usage: deploy.sh host}
  log "deploying to $target"
  rsync -a "$script_dir/dist/" "$target:/srv/app/"
}
main "$@"
```

CODE EXPLAINED

- → `script_dir` — find files next to the script, wherever it's run from
- → `mktemp -d` + `trap … EXIT` — private temp dir, always removed
- → `log` to stderr — keeps stdout clean for real output
- → `command -v` — POSIX way to test that a program exists
- → `${1:?usage}` — required argument with a built-in error
- → `main "$@"` — whole file is parsed before anything runs

INSTEAD OF → USE

AVOID	PREFER
`which x`	`command -v x`
`echo -e`	`printf`
`backticks`	`$( … )`
`for f in $(ls)`	`for f in *`
`[ $x = y ]`	`[[ $x == y ]]`
`expr / let`	`(( … ))`
`cat f | grep x`	`grep x f`

BASHISM → POSIX SH

BASH	POSIX SH
`[[ … ]]`	`[ … ]`
`==`	`=`
`arrays`	`"$@" / set --`
`{1..5}`	`seq / while loop`
`source f`	`. f`
`&> f`	`> f 2>&1`
`function f`	`f()`

MISSING IN BASH 3.2 (MACOS)

FEATURE	SINCE
`declare -A`	4.0
`mapfile / readarray`	4.0
`${v,,} ${v^^}`	4.0
`globstar, &>>, |&`	4.0
`declare -n, wait -n`	4.3
`${v@Q}`	4.4
`EPOCHSECONDS`	5.0

⚠ COMMON MISTAKES

- `#!/bin/sh` with `[[ ]]` or arrays: works where sh is bash, breaks on Debian (dash) and Alpine (busybox)
- Parsing `ls` output: breaks on spaces and newlines in names; use globs or `find -print0`
- `cd somewhere; rm -rf *` without checking `cd` succeeded; use `cd dir || exit`
- Hard-coding `/tmp/myfile`: collisions and security issues; use `mktemp`

✓ TIPS & BEST PRACTICES

- Quote everything, then remove quotes only where you want splitting
- Use `--` before untrusted filenames: `rm -- "$f"`
- Check the version you need: `(( BASH_VERSINFO[0] >= 4 )) || { echo 'need bash 4+' >&2; exit 1; }`
- Format with `shfmt -i 2 -w script.sh` for consistent style

QUICK SUMMARY

- ShellCheck — run on every script
- `#!/usr/bin/env bash` — be explicit about bash
- `command -v` / `printf` / `$( )` — portable choices
- `mktemp` + `trap` — safe temp files
- Quote every expansion

Quick Reference — Syntax & Expansion

QUOTING & OUTPUT · 03

SYNTAX	MEANING
`'text'`	literal, no expansion
`"text"`	expands $ $( ), no splitting
`$'a\tb'`	ANSI-C escapes
`\$`	escape one character
`printf '%s\n' x`	safe output
`printf -v v …`	format into a variable
`printf '%q' s`	shell-quote a string
`echo msg >&2`	write to stderr

REDIRECTION · 04

SYNTAX	MEANING
`> f / >> f`	write / append stdout
`< f`	stdin from file
`2> f`	stderr to file
`> f 2>&1`	both to file
`&> f / &>> f`	both (bash)
`a | b / a |& b`	pipe stdout / both
`<<EOF / <<'EOF'`	here-doc / literal
`<<< "$s"`	here-string

SPECIAL PARAMETERS · 08

SYNTAX	MEANING
`$0`	script name
`$1 … ${10}`	positional args
`$#`	arg count
`"$@"`	all args, separately
`$?`	last exit status
`$$ / $!`	shell PID / last bg PID
`PIPESTATUS`	statuses of last pipe
`$RANDOM`	0–32767

DEFAULTS & LENGTH · 09

SYNTAX	MEANING
`${v:-d}`	d if unset or empty
`${v-d}`	d if unset only
`${v:=d}`	default and assign
`${v:?msg}`	error if unset/empty
`${v:+a}`	a if set and non-empty
`${#v}`	string length
`${!v}`	indirect value
`${!pre@}`	names with prefix

SUBSTRINGS & CASE · 10

SYNTAX	MEANING
`${v:o:n}`	substring
`${v: -3}`	last 3 chars
`${v#p} / ${v##p}`	trim prefix short / long
`${v%p} / ${v%%p}`	trim suffix short / long
`${v/p/r}`	replace first
`${v//p/r}`	replace all
`${v^^} / ${v,,}`	upper / lower (4.0+)
`${v@Q}`	quoted copy (4.4+)

ARRAYS · 11–12

SYNTAX	MEANING
`a=(x y z)`	indexed array
`declare -A h`	associative (4.0+)
`"${a[@]}"`	all elements
`${#a[@]}`	count
`${!a[@]}`	indices / keys
`a+=(w)`	append
`${a[@]:1:2}`	slice
`[[ -v h[k] ]]`	key exists

GLOBS & BRACES · 13–14

SYNTAX	MEANING
`* / ?`	any string / one char
`[a-z] / [!x]`	set / negated set
`**/`	recursive (globstar)
`!(p) / @(a|b)`	extglob not / one of
`{a,b,c}`	list
`{1..9..2}`	range with step
`f{,.bak}`	f f.bak
`~ / ~-`	home / previous dir

TESTS · 15

SYNTAX	TRUE IF
`[[ -e f ]]`	exists
`[[ -f f ]] / -d`	file / directory
`[[ -z s ]] / -n`	empty / non-empty
`[[ s == p* ]]`	glob match
`[[ s =~ re ]]`	regex (BASH_REMATCH)
`[[ a -lt b ]]`	integer compare
`[[ a -nt b ]]`	a newer than b
`[[ -v name ]]`	variable is set

ARITHMETIC · 17

SYNTAX	MEANING
`$(( a + b ))`	integer result
`(( a > b ))`	numeric test
`(( i++ ))`	increment
`** / %`	power / remainder
`c ? x : y`	ternary
`16#ff / 2#101`	base#number
`10#08`	force decimal
`bc -l / awk`	decimals

⚠ COMMON MISTAKES

- Unquoted `$var` or `${arr[@]}` : split and globbed
- `> f 2>&1` and `2>&1 > f` are different
- `{1..$n}` never expands; braces run first

✓ TIPS & BEST PRACTICES

- Quote by default: `"$v"`, `"$(cmd)"`, `"${a[@]}"`
- Use `[[ ]]` for strings, `(( ))` for numbers
- `declare -p var` shows exactly what's stored

QUICK SUMMARY

- `${v:-d}` / `${v%p}` / `${v//p/r}` — default, trim, replace
- `"${a[@]}"` — every element safely
- `2>&1` / `<<EOF` — redirect and inline input

Quick Reference — Scripting & Interactive

CONTROL FLOW · 16, 18

SYNTAX	MEANING
`if cmd; then … fi`	branch on status
`elif / else`	more branches
`case $x in p) ;; esac`	pattern match
`for x in list`	loop over words
`for ((i=0;i<n;i++))`	counting loop
`while / until cmd`	loop on status
`while IFS= read -r l`	read lines
`break / continue`	loop control

FUNCTIONS · 19

SYNTAX	MEANING
`f() { …; }`	define
`f a b`	call with args
`local v`	function scope
`return n`	exit status 0–255
`out=$(f)`	capture output
`local -n r=$1`	nameref (4.3+)
`declare -f f`	show definition
`main "$@"`	entry-point pattern

INPUT & ARGUMENTS · 20–21

SYNTAX	MEANING
`read -r v`	one line
`read -rp 'Q? ' v`	with prompt
`read -rsp … pw`	silent input
`read -ra arr`	split into array
`mapfile -t a < f`	lines to array
`getopts "vo:" opt`	short options
`shift $((OPTIND-1))`	drop parsed options
`--`	end of options

ERRORS & TRAPS · 22

SYNTAX	MEANING
`set -euo pipefail`	strict mode
`cmd || true`	allow failure
`trap 'f' EXIT`	always run f
`trap 'f' ERR`	on failure
`trap 'f' INT TERM`	on Ctrl-C / kill
`set -x / set +x`	trace on / off
`bash -n s.sh`	syntax check
`shellcheck s.sh`	lint

JOBS & SUBSTITUTION · 23

SYNTAX	MEANING
`cmd &`	background
`jobs / fg / bg`	manage jobs
`wait / wait -n`	all / next (4.3+)
`disown / nohup`	survive logout
`( … )`	subshell
`{ …; }`	group, same shell
`$(cmd)`	output as string
`<(cmd)`	output as a file

OPTIONS & PROMPT · 02, 24

SYNTAX	MEANING
`set -o / +o x`	POSIX option on / off
`shopt -s / -u x`	bash option on / off
`shopt -s globstar`	enable **
`shopt -s nullglob`	empty when no match
`alias n='cmd'`	shortcut
`\cmd`	skip alias
`PS1='\u@\h:\w\$ '`	prompt
`pushd / popd / cd -`	dir navigation

READLINE KEYS · 06

KEYS	ACTION
`Ctrl-A / Ctrl-E`	line start / end
`Alt-B / Alt-F`	word back / forward
`Ctrl-W / Ctrl-U`	delete word / to start
`Ctrl-K / Ctrl-Y`	cut to end / paste
`Ctrl-R`	search history
`Alt-.`	last argument
`Ctrl-X Ctrl-E`	edit in $EDITOR
`Tab Tab`	list completions

HISTORY · 05

SYNTAX	MEANING
`!!`	previous command
`sudo !!`	again, with sudo
`!$ / !^ / !*`	last / first / all args
`!str`	last cmd starting str
`!n`	command number n
`^old^new`	rerun with fix
`!!:p`	print, don't run
`history -a`	save now

STARTUP FILES · 01

FILE	READ BY
`/etc/profile`	login shells
`~/.bash_profile`	login shells
`~/.profile`	login, if no .bash_profile
`~/.bashrc`	interactive non-login
`~/.bash_logout`	login shell exit
`~/.inputrc`	Readline programs
`$BASH_ENV`	scripts
`source f`	run f in this shell

VERSION MATRIX (✔ = AVAILABLE) · 25

FEATURE	3.2	4.0	4.3	4.4	5.1
`declare -A`	✖	✔	✔	✔	✔
`mapfile / readarray`	✖	✔	✔	✔	✔
`${v,,} ${v^^}`	✖	✔	✔	✔	✔
`globstar **`	✖	✔	✔	✔	✔
`&>> |&`	✖	✔	✔	✔	✔

VERSION MATRIX, CONTINUED

FEATURE	3.2	4.0	4.3	4.4	5.1
`declare -n / wait -n`	✖	✖	✔	✔	✔
`${v@Q} / local -`	✖	✖	✖	✔	✔
`inherit_errexit`	✖	✖	✖	✔	✔
`EPOCHSECONDS (5.0)`	✖	✖	✖	✖	✔
`${v@U} / SRANDOM`	✖	✖	✖	✖	✔

⚠ COMMON MISTAKES

- `cmd | while read` loses variables (subshell)
- `local v=$(cmd)` hides `cmd`'s failure
- macOS `/bin/bash` is 3.2: check the matrix

✓ TIPS & BEST PRACTICES

- `#!/usr/bin/env bash` picks up a newer Homebrew bash
- Clean up with `trap 'rm -rf "$tmp"' EXIT`
- Check versions: `(( BASH_VERSINFO[0] >= 4 ))`

QUICK SUMMARY

- `if` / `case` / `for` / `while` — control flow
- `set -euo pipefail` + `trap` — safe scripts
- `&` + `wait` — run jobs in parallel